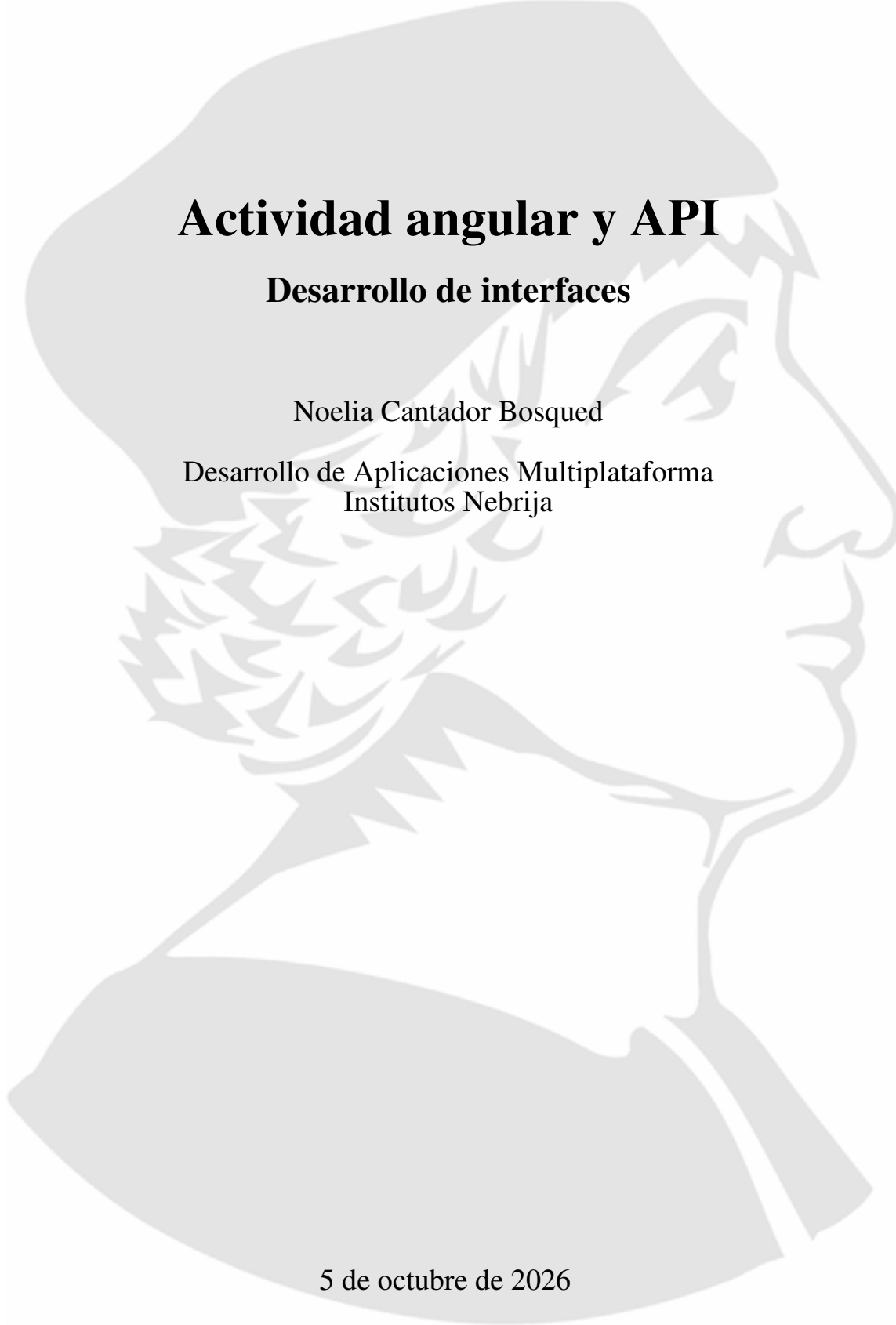




Actividad angular y API

Desarrollo de interfaces

Noelia Cantador Bosqued

Desarrollo de Aplicaciones Multiplataforma
Institutos Nebrija

5 de octubre de 2026

Índice

1. Estructura del Proyecto y Filosofía Standalone	2
1.1. ¿Qué significa Angular Standalone y por qué lo usamos?	2
1.2. Árbol de directorios implementado	2
2. Configuración Inicial y Errores Resueltos	3
2.1. Corrección de la errata tipográfica en el modelo	3
2.2. Adaptación de importaciones a Ionic v9	3
2.3. El bloqueo de refresco en la llamada a la API (Detección Zoneless)	3
3. Modelado y Servicio de Datos (API REST)	3
3.1. Modelo de datos: product.model.ts	3
3.2. Servicio de consumo con inject(): product.service.ts	4
4. Diseño de Vistas y Navegación	4
4.1. Enrutamiento: app.routes.ts	4
4.2. Página de Inicio (inicio.page)	5
4.3. Página de Productos (productos.page)	5
5. Control de Versiones y Despliegue	5
5.1. Git y GitHub en la rama desarrollo	5
5.2. Despliegue en Vercel y resolución de rutas SPA	5
6. Checklist de Cumplimiento de la Actividad	6
7. Mejoras y Ampliaciones del Proyecto	6
7.1. Galería de productos locales: Colección Juicy Couture en cuadrícula	6
7.2. Ampliación de campos en la tabla de la API REST	7
7.3. Optimización de la detección de cambios (Arquitectura Zoneless)	8
7.4. Nueva página corporativa: “About” (/about)	8
7.5. Optimización de la navegación y experiencia de usuario (UX)	8
8. Peticiones Adicionales, Retos Implementados y Flujo de Trabajo	8
8.1. Ampliaciones en el Catálogo de Productos y Métricas	9
8.2. Nueva Página “About” (/about) y Navegación desde Inicio	9
8.3. Resolución de Incidencias Técnicas (Troubleshooting)	9
8.4. Reto 1: Implementación de Paginación en Productos	10
8.5. Implementación del Modo Oscuro (Dark Mode)	10
8.6. Reto Brutal: Componente Visual Atractivo (Dashboard y Cards)	11
8.7. Arquitectura Standalone: ¿Por qué existe app.config.ts?	11
8.8. Flujo de Trabajo Profesional con Git y Vercel (Git Flow)	12
8.9. Autoevaluación y Aspectos a Consolidar	12

1. Estructura del Proyecto y Filosofía Standalone

1.1. ¿Qué significa Angular Standalone y por qué lo usamos?

En versiones tradicionales de Angular, cualquier componente, directiva o tubería (*pipe*) requería obligatoriamente ser declarado en un módulo `@NgModule` (`app.module.ts`), lo cual provocaba sobrecarga de código y dependencias innecesarias en cascada.

En **Angular Standalone**:

- La aplicación se configura en `src/app/app.config.ts` mediante la función `bootstrapApplication(AppComponent, appConfig)` ejecutada en `src/main.ts`.
- Cada componente declara en su metadato `@Component({ standalone: true, imports: [...] })` exactamente lo que necesita (botones, tarjetas, pipes, etc.).
- No existe ningún archivo `app.module.ts`.

1.2. Árbol de directorios implementado

El código se ha estructurado de forma limpia y modular tal y como pedía la práctica:

```
src/  
  app/  
    models/  
      product.model.ts  
    services/  
      product.service.ts  
    pages/  
      inicio/  
        inicio.page.html  
        inicio.page.scss  
        inicio.page.ts  
      productos/  
        productos.page.html  
        productos.page.scss  
        productos.page.ts  
    app.component.html  
    app.component.ts  
    app.routes.ts  
    app.config.ts
```

2. Configuración Inicial y Errores Resueltos

Durante el proceso de desarrollo y ensamblado de los ficheros de la actividad se produjeron varios errores típicos de configuración y sintaxis que detecté y corregí:

2.1. Corrección de la errata tipográfica en el modelo

Problema: El modelo se había nombrado `product.models.ts` (con plural “s”), mientras que el servicio y las páginas importaban desde `../models/product.model`, lanzando error `TS2307: Cannot find module.`

Solución: Renombré el fichero a `product.model.ts`.

2.2. Adaptación de importaciones a Ionic v9

Problema: El manual de clase contemplaba Ionic 7/8 donde los componentes standalone se importaban de `@ionic/angular/standalone`. Al haber generado el proyecto con la versión actual (**Ionic 9**), esa subruta fue retirada porque todo el paquete raíz `@ionic/angular` es standalone por defecto. La compilación fallaba indicando “*The path ./standalone is not exported*”.

Solución: Actualicé las importaciones a:

```
import { IonHeader, IonToolbar, IonTitle, IonContent, IonButton } from '@ionic/angular';
```

2.3. El bloqueo de refresco en la llamada a la API (Detección Zoneless)

Problema: Al acceder a la vista de productos, la pantalla se quedaba permanentemente con el mensaje “*Cargando productos...*” sin renderizar los datos. Esto ocurría porque Angular 22 venía configurado sin `zone.js` (*Zoneless*), por lo que al completarse la suscripción asíncrona de RxJS (`.subscribe()`), Angular no se enteraba del cambio de estado.

Solución: Inyecté `ChangeDetectorRef` en `ProductosPage` y añadí `provideZonelessChangeDetection()` en `app.config.ts`. Tras recibir los datos, se ejecuta `this.cdr.markForCheck()`, actualizando la vista inmediatamente.

3. Modelado y Servicio de Datos (API REST)

3.1. Modelo de datos: `product.model.ts`

Definé las interfaces tipadas que representan fielmente el objeto JSON devuelto por la API de DummyJSON:

```
export interface Product {  
  id: number;  
  title: string;  
  description: string;  
  category: string;  
  price: number;  
}
```

```
    discountPercentage: number;
    rating: number;
    stock: number;
    brand: string;
    thumbnail: string;
  }

  export interface ProductsResponse {
    products: Product[];
    total: number;
    skip: number;
    limit: number;
  }
}
```

3.2. Servicio de consumo con inject(): product.service.ts

En lugar del constructor clásico, utilicé la función moderna `inject()` de Angular:

```
import { Injectable, inject } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable } from 'rxjs';
import { ProductsResponse } from '../models/product.model';

@Injectable({
  providedIn: 'root'
})
export class ProductService {
  private http = inject(HttpClient);
  private apiUrl = 'https://dummyjson.com/products';

  getProducts(): Observable<ProductsResponse> {
    return this.http.get<ProductsResponse>(this.apiUrl);
  }
}
```

4. Diseño de Vistas y Navegación

4.1. Enrutamiento: app.routes.ts

Configuré rutas limpias con redirección por defecto y comodín (*wildcard*):

- '' → Redirige a /inicio.
- /inicio → Carga diferida de InicioPage.
- /productos → Carga diferida de ProductosPage.
- ** → Redirige a /inicio.

4.2. Página de Inicio (inicio.page)

Incluye la cabecera con el título de DAM2, un texto descriptivo y el botón con la directiva RouterLink que navega hacia la ruta /productos.

4.3. Página de Productos (productos.page)

En esta página integré dos componentes visuales esenciales:

1. **Galería de bolsos Juicy Couture en formato cuadrícula (“cuadraditos”):** Moví las cuatro fotografías locales (bolso1.jpg, bolso2.jpg, bolso3.jpg, bolso4.jpg) a src/assets/images/. En la plantilla se muestran en tarjetas cuadradas (aspect-ratio: 1/1) con su título, descripción y precio formateado en euros con CurrencyPipe.
2. **Tabla de la API REST DummyJSON:** Control de flujo moderno de Angular con directivas nativas @if (loading), @if (error) con botón de reintento, y @if (!loading && !error) con un bucle @for (product of products; track product.id) que renderiza la tabla con ID, miniatura, título, categoría, marca, precio, valoración y stock.

5. Control de Versiones y Despliegue

5.1. Git y GitHub en la rama desarrollo

1. Creé y cambié a la rama solicitada:

```
git checkout -b desarrollo
```

2. Vinculé el repositorio remoto en GitHub:

```
git remote add origin git@github.com:NoeCB/dam2-productos.git
```

3. Confirmé y subí los cambios:

```
git add .  
git commit -m "Aplicacion con productos"  
git push -u origin desarrollo
```

5.2. Despliegue en Vercel y resolución de rutas SPA

Las aplicaciones Angular Standalone son aplicaciones de página única (*Single Page Applications*). Si un usuario accede o refresca directamente la URL /productos en Vercel, el servidor puede devolver un error 404 porque no existe un archivo físico productos.html.

Para resolver esto, creé el archivo de configuración vercel.json en la raíz:

```
{  
  "cleanUrls": true,  
  "rewrites": [  
    { "source": "/(.*)", "destination": "/index.html" }  
  ]  
}
```

```
]
}
```

En Vercel se configuró el comando de construcción `npm run build` y la carpeta de salida `www`.

6. Checklist de Cumplimiento de la Actividad

Categoría	Requisito del profesor	Estado
Angular Standalone	Eliminación total de <code>app.module.ts</code>	Cumplido
	Componentes y raíz configurados con <code>standalone: true</code>	Cumplido
	Imports declarados explícitamente en cada componente	Cumplido
	Uso de <code>app.config.ts</code> con <code>provideHttpClient()</code>	Cumplido
API REST	Consumo de https://dummyjson.com/products	Cumplido
	Interfaz <code>Product</code> y <code>ProductsResponse</code>	Cumplido
	Inyección mediante <code>inject(ProductService)</code>	Cumplido
Navegación	Rutas <code>/inicio</code> , <code>/productos</code> y redirecciones	Cumplido
	Navegación de ida (<code>IonButton</code>) y vuelta (<code>IonBackButton</code>)	Cumplido
Estados de carga	Spinner de carga (<code><ion-spinner></code>)	Cumplido
	Tarjeta de error con botón de reintento	Cumplido
Git y Despliegue	Repositorio en GitHub en la rama <code>desarrollo</code>	Cumplido
	Despliegue en Vercel con <code>rewrites SPA</code> en <code>vercel.json</code>	Cumplido

Tabla 1

Lista de comprobación y verificación final de la práctica.

7. Mejoras y Ampliaciones del Proyecto

Tras completar los requisitos funcionales mínimos exigidos en la actividad guiada, se han implementado una serie de mejoras tanto a nivel de funcionalidad y lógica de negocio como de diseño de interfaz de usuario (UI/UX) y arquitectura técnica.

7.1. Galería de productos locales: Colección Juicy Couture en cuadrícula

Como ampliación al catálogo proveniente de la API externa, se ha diseñado una sección de productos destacados utilizando imágenes propias almacenadas localmente:

- **Gestión de recursos estáticos:** Se ha creado el directorio `src/assets/images/` para albergar las fotografías de los artículos (`bolso1.jpg`, `bolso2.jpg`, etc.).
- **Diseño de tarjetas cuadradas (“cuadraditos”):** Mediante CSS Grid (`.cuadraditos-grid`) se ha creado un diseño adaptativo (*responsive*) donde cada producto se presenta dentro de

una `<ion-card>` con relación de aspecto cuadrada (`aspect-ratio: 1 / 1`), imagen ajustada con `object-fit: cover`, efecto de elevación (*hover*) y tipografía destacada.

- **Modelado y formateo:** Se definió una interfaz TypeScript JuicyBag y se aplicó la tubería CurrencyPipe de Angular para formatear los importes en euros de forma estándar (89,99 €, etc.).

7.2. Ampliación de campos en la tabla de la API REST

Se ha profundizado en el consumo del objeto devuelto por <https://dummyjson.com/products>, agregando nuevas columnas calculadas y atributos físicos a la tabla:

1. **Dimensiones físicas (Ancho × Alto):** Se amplió la interfaz Product incorporando el objeto anidado:

```
export interface ProductDimensions {
  width: number;
  height: number;
  depth?: number;
}
```

En la plantilla se renderiza condicionalmente el tamaño del artículo:

```
<td>
  @if (product.dimensions) {
    {{ product.dimensions.width }} x {{ product.dimensions.height }} cm
  } @else {
    N/D
  }
</td>
```

2. **Cálculo del Stock Valorado con descuento:** Se añadió un método en el controlador que calcula el valor total del inventario disponible en almacén para cada referencia, deduciendo el porcentaje de oferta aplicable:

$$\text{Stock Valorado} = \text{Stock} \times \left(\text{Precio} \times \left(1 - \frac{\text{Descuento}}{100} \right) \right)$$

```
calcularStockValorado(product: Product): number {
  const descuento = product.discountPercentage ? product.discountPercentage / 100 : 0;
  return product.stock * product.price * (1 - descuento);
}
```

En la vista se muestra el total formateado junto a un distintivo con el tanto por ciento de rebaja aplicado (-15.4%).

7.3. Optimización de la detección de cambios (Arquitectura Zoneless)

Debido a que Angular 22 opera sin la librería tradicional `zone.js`, la actualización del DOM tras la llegada asíncrona de datos desde la API no se disparaba automáticamente. Se resolvieron estos bloqueos mediante dos intervenciones clave:

- Configuración de `provideZonelessChangeDetection()` en el archivo de arranque `app.config.ts`.
- Inyección de `ChangeDetectorRef` en `ProductosPage` e invocación explícita de `this.cdr.markForCheck()` en los *callbacks* de éxito y error de la petición HTTP.
- Implementación del ciclo de vida propio de Ionic `ionViewWillEnter()` para garantizar la recarga automática cada vez que el usuario navega a la página.

7.4. Nueva página corporativa: “About” (/about)

Se ha creado una tercera vista independiente y completamente modular para dar una identidad empresarial al proyecto:

- **Componente Standalone:** Creado en `src/app/pages/about/` y registrado en `app.routes.ts` con carga perezosa (*lazy loading*).
- **Reseña empresarial:** Presenta la marca *Juicy Couture Store & DAM2 Tech*, combinando la historia del catálogo con los valores de desarrollo frontend multiplataforma.
- **Enlace al perfil de GitHub:** Incorpora un acceso directo verificado al perfil personal del desarrollador (<https://github.com/NoeCB>) para facilitar la revisión del código fuente.

7.5. Optimización de la navegación y experiencia de usuario (UX)

Se enriqueció el flujo de navegación entre pantallas:

- En la página de **Inicio**, se ofrecen dos botones diferenciados: acceso al catálogo completo (`/productos`) y acceso a la información corporativa (`/about`).
- En la página de **Productos**, además del botón nativo de retroceso en el encabezado (`ion-back-button`), se han colocado botones accesibles tanto al inicio como al pie de página para regresar rápidamente a `/inicio`.

8. Peticiones Adicionales, Retos Implementados y Flujo de Trabajo

En este apartado se detalla la resolución de las nuevas funcionalidades requeridas por el cliente, los dos retos técnicos implementados en la aplicación, el flujo profesional de control de versiones con Git y Vercel, y la justificación arquitectónica del proyecto.

8.1. Ampliaciones en el Catálogo de Productos y Métricas

Se ha profundizado en el procesamiento de la respuesta de la API REST (<https://dummyjson.com/products>):

- **Dimensiones de ancho y alto:** Se amplió la interfaz Product incorporando la estructura `dimensions: { width, height, depth }`. En la tabla se renderiza la columna con el formato:

`{{ product.dimensions.width }} × {{ product.dimensions.height }} cm`

- **Cálculo del Stock Valorado con descuento:** Para cada referencia se calcula el valor económico real en almacén aplicando la rebaja comercial:

$$\text{Stock Valorado} = \text{unidades} \times \text{precio} \times \left(1 - \frac{\text{descuento}}{100}\right)$$

Este cálculo se realiza mediante el método `calcularStockValorado(product)` y se muestra formateado con la tubería `currency: 'EUR'`.

- **Botones de retorno a Inicio:** Se han incorporado botones con la directiva `routerLink=/inicio` tanto en la barra superior (`<ion-header>`) como al inicio y pie de página, facilitando la navegación sin depender del botón atrás del navegador.

8.2. Nueva Página “About” (/about) y Navegación desde Inicio

Se ha creado un componente Standalone en `src/app/pages/about/` registrado en el enrutador `app.routes.ts`:

- **Navegación principal:** La página de inicio (`/inicio`) cuenta ahora con dos accesos directos: botón primario hacia `/productos` y botón secundario hacia `/about`.
- **Reseña corporativa:** Describe la empresa (*Juicy Couture Store & DAM2 Tech*), detallando su trayectoria, enfoque en moda y accesorios, y su compromiso con el desarrollo de software frontend moderno.
- **Enlace a GitHub:** Incluye un botón interactivo que redirige al perfil oficial de GitHub de la desarrolladora (<https://github.com/NoeCB>).

8.3. Resolución de Incidencias Técnicas (Troubleshooting)

Durante el desarrollo se detectaron y solventaron dos problemas críticos:

Incidencia 1: Bloqueo de la vista en “Cargando productos...” (Angular Zoneless)

- **Causa:** Al generarse el proyecto con Angular 22, este prescinde de la librería antigua `zone.js` (*arquitectura Zoneless*). Por tanto, cuando la petición asíncrona de RxJS (`http.get().subscribe()`) recibía los datos, el ciclo de detección de cambios de Angular no se enteraba y la pantalla permanecía congelada en el spinner.
- **Solución:** Se configuró `provideZonelessChangeDetection()` en `app.config.ts` y se inyectó `ChangeDetectorRef` en el componente de productos, invocando `this.cdr.markForCheck()` al recibir la respuesta HTTP para obligar al DOM a redibujarse.

Incidencia 2: Error de resolución con “@ionic/angular/standalone”

- **Causa:** El guión de la práctica utilizaba la ruta `@ionic/angular/standalone`, válida en Ionic 7 y 8. Sin embargo, el CLI instaló la versión actual (**Ionic 9**), en la cual el equipo de Ionic eliminó dicha subruta porque el paquete raíz `@ionic/angular` ya es 100% Standalone por defecto. Esto provocaba un fallo fatal de compilación: *“The path ./standalone is not exported by package @ionic/angular”*.
- **Solución:** Se sustituyeron todas las importaciones para obtener los componentes directamente desde `@ionic/angular`.

8.4. Reto 1: Implementación de Paginación en Productos

Para optimizar el rendimiento y evitar un scroll infinito con los 30 productos de la API, se ha implementado un sistema de paginación reactivo en el cliente:

- **Parámetros de control:** Se establecieron las variables `currentPage = 1` y un tamaño de página de `pageSize = 6`.
- **Propiedades computadas:**
 - `totalPages`: Calcula el número total de páginas mediante `[total/pageSize]`.
 - `paginatedProducts`: Aplica un corte con `.slice()` para obtener exclusivamente el subconjunto de productos correspondiente a la página activa.
- **Controles en la interfaz:** Botones de “← Anterior” y “Siguiente →” (que se deshabilitan en los extremos) junto con una botonera numérica interactiva que resalta la página actual y actualiza la vista con `markForCheck()`.

8.5. Implementación del Modo Oscuro (Dark Mode)

Atendiendo a la solicitud del cliente, se desarrolló un conmutador de tema visual accesible desde la barra superior de todas las pantallas (`/inicio`, `/productos` y `/about`):

- **Servicio ThemeService:** Centraliza el estado en `src/app/services/theme.service.ts`, detecta la preferencia del sistema operativo con `window.matchMedia(' (prefers-color-scheme: dark)')` y persiste la elección en el almacenamiento local (`localStorage`).

- **Integración con Ionic 9:** Al alternar el botón (Claro / Oscuro), se añade o retira la clase oficial `.ion-palette-dark` en `document.documentElement`, adaptando automáticamente paletas de color, tarjetas, tablas y fondos.

8.6. Reto Brutal: Componente Visual Atractivo (Dashboard y Cards)

En lugar de restringir la visualización de la API a una simple tabla convencional, se implementó un ****Dashboard interactivo con selector de vistas****:

1. **Panel de Métricas (KPI Cards):** Tarjetas en cuadrícula superior que calculan y resumen en tiempo real:
 - Total de productos cargados.
 - Unidades totales en stock físico.
 - Valoración monetaria total del inventario.
 - Porcentaje medio de descuento de todo el catálogo.
2. **Selector de Modo de Visualización:** Permite al usuario alternar mediante botones entre:
 - **Vista Tarjetas (Dashboard):** Tarjetas interactivas con insignia de categoría (`ion-badge`), imagen con zoom progresivo al pasar el cursor (*hover effect*), precio de venta recomendado tachado junto al precio final rebajado, desglose de stock valorado y dimensiones físicas.
 - **Vista Tabla Detallada:** Mantiene la representación tabular para auditoría formal de datos.

8.7. Arquitectura Standalone: ¿Por qué existe `app.config.ts`?

Sí, el fichero existe en la ruta `src/app/app.config.ts`.

Justificación técnica: En Angular clásico, los proveedores de servicios (`providers`) y la configuración raíz se definían dentro del decorador `@NgModule` del archivo `app.module.ts`. Al prescindir de los módulos en la arquitectura **Standalone**, el framework requiere un mecanismo desacoplado para configurar los servicios globales antes de iniciar el ciclo de vida de la aplicación.

Por ello existe `app.config.ts`, donde se define un objeto de tipo `ApplicationConfig` que registra:

- `provideHttpClient(withFetch())`: Permite que los servicios inyecten el cliente HTTP utilizando la API nativa `fetch`.
- `provideZonelessChangeDetection()`: Habilita el nuevo motor de detección de cambios sin sobrecarga de zonas.
- Proveedores de enrutamiento y estrategias de Ionic.

Posteriormente, dicho objeto se pasa como argumento a `bootstrapApplication(AppComponent, appConfig)` en `src/main.ts`.

8.8. Flujo de Trabajo Profesional con Git y Vercel (Git Flow)

¿Por qué Vercel debe mostrar la rama main? En ingeniería de software y entornos de integración continua (CI/CD), la rama `main` representa la versión de ****producción estable, probada y validada****, mientras que la rama `desarrollo` alberga las características en construcción. Si Vercel desplegara directamente cada cambio inestable de desarrollo, los usuarios finales podrían experimentar fallos no verificados.

Procedimiento para fusionar de branch desarrollo a main: El flujo de trabajo correcto para promocionar los cambios validados es:

1. Asegurar que todos los cambios locales están confirmados en la rama de desarrollo:

```
git checkout desarrollo
git status
git add .
git commit -m "feat: implementacion final de productos, retos y
about"
git push origin desarrollo
```

2. Cambiar a la rama principal (main):

```
git checkout main
```

3. Fusionar los cambios probados de la rama desarrollo:

```
git merge desarrollo
```

4. Publicar la rama `main` actualizada en GitHub:

```
git push origin main
```

Al detectar el nuevo commit en `main`, el webhook de Vercel iniciará automáticamente una nueva construcción (*build*) y desplegará la versión final en el dominio público de producción.

8.9. Autoevaluación y Aspectos a Consolidar

- **Objetivos completados al 100 %:** Implementación de Standalone sin módulos, consumo e inyección tipada de la API REST, enrutamiento lazy loading, control de flujo con sintaxis moderna (`@if` y `@for`), cálculo dinámico de stock valorado, paginación reactiva, conmutador de modo oscuro y Dashboard visual de tarjetas.
- **Aspectos a repasar:** Profundizar en la paginación a nivel de servidor mediante parámetros en la URL (`?limit=10&skip=10`) frente a la paginación en cliente utilizada, y practicar la resolución de conflictos en Git durante operaciones de rebase o merge en equipos concurrentes.